Copilot Architecte

Une équipe de 12 agents spécialisés pour GitHub Copilot CLI, dont un expert du scraping et un pentester, qui travaille en autonomie sur votre PC. Cette version n'a pas de Gardien : aucune commande n'est contrôlée avant de s'exécuter.

Installer en confiant le travail à une IA

Vous n'avez pas besoin de lancer les commandes vous-même : une IA capable d'agir sur votre PC les exécute et vous guide.

1. Préparez
Lancez votre IA en mode sans restriction : `copilot --yolo`, `claude --dangerously-skip-permissions`, `codex --yolo`.

2. Vérifiez l'IA
Elle doit avoir accès à vos fichiers et à un terminal, sous Windows.

3. Collez le prompt
Copiez le texte ci-dessous. L'IA installe, vérifie, puis vous fait un rapport.

Le prompt à coller

```
Installe Copilot Architecte sur ce PC, en toute autonomie et jusqu'au bout.
Clone https://github.com/TON-COMPTE/copilot-architecte dans mon dossier personnel, ou télécharge son ZIP si Git est absent.
Place-toi dans ce dossier, lis AGENTS.md et applique-le étape par étape.
Si ta session demande des autorisations, demande-moi une seule fois de te relancer en mode sans restriction.
Ne me demande jamais une clé dans la conversation.
```

Si le dossier est déjà sur votre PC, lancez l'IA depuis ce dossier et écrivez simplement : `Installe.` Sans IA : double-cliquez sur `installer.cmd`, ou suivez `INSTALL_MANUEL.md`.

> **Lisez ceci avant d'accepter.** Une fois installé, Copilot agit avec tous les droits de votre compte Windows, sans rien vous demander. Il peut supprimer des fichiers, utiliser vos identifiants et envoyer des données sur le réseau. Installez le kit sur votre propre poste, ou sur celui d'une personne informée, et évitez une machine contenant des accès de production ou des données de tiers.

Ce qui reste à votre charge

- **Avoir Copilot :** un abonnement GitHub Copilot et une session ouverte avec `copilot login`.
- **Taper vos clés de service :** dont la clé TypeSafe de JEV, dans votre terminal avec `.\mcp-scraping.ps1`, jamais dans la conversation avec l'IA.
- **Vous connecter aux services :** dans Copilot, ouvrez `/mcp` pour Firecrawl, TinyFish, Olostep et Apify.

Ce que l'IA installe

Tout va dans `~/.copilot`, le dossier de Copilot dans votre profil Windows. Vos agents, skills et réglages sans rapport sont conservés, et chaque fichier modifié est d'abord sauvegardé.

Élément	Rôle
12 agents et 28 skills	L'équipe et ses méthodes de travail
`copilot-instructions.md`	Les règles communes à tous les agents ; vos propres instructions sont conservées
`settings.json`	Le réglage en pleine autonomie, fusionné avec vos réglages existants
`mcp-config.json`	Les serveurs de scraping et de décision
Moteur de scraping et Python	Environnement Python séparé, téléchargement d'environ 200 Mo
Navigateur Jev Ultrafast	Environnement Python séparé, version figée, profil Chrome dédié
Profil PowerShell	Les commandes `cop`, `copr` et `cop-pentest`
Ligne d'état	`statusline.ps1` : modèle, contexte, requêtes et durée en bas de la session

Comment savoir que l'installation a réussi

L'IA lance `.\verifier.ps1`. Chaque contrôle affiche OK ou KO et le dernier message doit être `RESULT: all checks OK`. Ensuite, deux essais confirment le fonctionnement réel. Ils consomment quelques crédits.

```
# 1. La délégation fonctionne : smoke.md doit montrer au moins deux sous-agents lancés
copr "Ajoute une fonction utilitaire avec ses tests. Délègue l'implémentation au spécialiste concerné
puis la revue à code-review, et cite les agents réellement appelés." --share smoke.md

# 2. Le scraping fonctionne de bout en bout
copr "Délègue à scraper : extrais le titre de https://example.com avec un outil MCP, range le résultat
selon la skill data-organization et cite l'outil utilisé."
```

L'utiliser au quotidien

Commande	Effet
`cop`	Ouvre une session : l'Architect reçoit vos demandes et les confie aux bons spécialistes
`cop -i "votre demande"`	Pareil, en lançant directement la demande
`copr "votre demande"`	Travaille jusqu'au bout sans poser de question, puis s'arrête
`copr "votre demande" --fleet`	Pareil, avec des sous-agents en parallèle
`cop-pentest`	Session d'attaque et d'audit de vos propres applications
`/usage /mcp /tasks`	Dans une session : consommation, état des serveurs, sous-agents en cours

Vous parlez normalement. Par exemple : *« Récupère tous les produits de ce catalogue avec prix et disponibilité, et donne-moi un fichier Excel. »*

Ce que fait le kit

L'Architect comprend la demande, planifie, délègue à un seul responsable par domaine, contrôle les résultats et rend un rapport. Chaque agent doit distinguer ce qu'il a vérifié de ce qu'il suppose, et ne jamais présenter un test comme réussi s'il ne l'a pas lancé.

Agent	Domaine
`architect`	Coordination, plan, consolidation et rapport final
`scraper`	Collecte de données web à toute échelle, rangement et contrôle des données
`pentest`	Attaque de vos applications, preuve des failles, nouveau test après correction
`frontend` / `backend`	Interfaces et intégration ; API et logique serveur, en Java comme dans toute autre pile
`database`	Schémas, SQL, migrations et performances
`devops`	CI/CD, Docker, environnements et déploiement
`security`	Menaces, vulnérabilités, secrets et dépendances
`testing-strategy` / `qa-automation`	Décider quoi tester ; écrire et exécuter les tests automatisés
`code-review`	Relecture indépendante du travail des autres agents
`documentation`	README, guides et documentation vérifiés sur le comportement réel

28 skills (méthodes chargées à la demande) : angular, api-testing, clean-code, confluence, crawlbase, data-organization, database, decision-support, docker, file-intelligence, firecrawl, github, gitlab-ci, java, jev-browser, jira, knowledge-management, nextjs, olostep, pentest, qa-automation, security, source-access, spring-boot, testing-strategy, tinyfish, ui-design, web-scraping.

Ce qui est débridé

Par défaut, Copilot demande souvent votre accord et s'arrête régulièrement. Avec le kit, il travaille seul jusqu'à la fin de la tâche.

Capacité	Effet
Permissions totales	Tous les outils, tous les chemins et toutes les URL, sans confirmation
Mode interactif	`cop` peut demander s'il faut changer le niveau d'effort. `copr` reste en autopilot, sans question
Puissance	Effort `high` avec un modèle fixé (le mode Auto choisit lui-même), fonctions expérimentales, mémoire entre les sessions
GitHub	Tous les outils du serveur GitHub, au lieu du sous-ensemble par défaut
Verrous retirés	Pas de bac à sable, pas d'interdiction dans vos réglages, scripts PowerShell autorisés
Second avis	Critique automatique par une autre famille de modèles aux moments importants
Continuité	Bascule sur un autre modèle en cas de limite de débit, PC maintenu éveillé pendant le travail

Aucune limite n'est posée à l'autonomie. Aucun plafond de crédits n'est posé : suivez votre consommation avec `/usage`. Pour imposer un effort, relancez l'installation avec `-Model <nom> -Effort <niveau>`.

L'agent Scraper

Il mène la collecte de bout en bout : il cherche la voie la plus directe, teste sur quelques pages, lance le volume complet, contrôle les résultats et livre des données rangées. Il choisit toujours l'outil le moins coûteux qui fonctionne.

Situation	Outil choisi
API, sitemap, JSON caché	Accès direct ou moteur local
Très grand volume, HTML simple	Moteur local : gratuit, reprend après un arrêt
Site en JavaScript	Firecrawl, Crawlbase
Clics, formulaires, parcours	Playwright, TinyFish
Grands lots sans infrastructure	Olostep, jusqu'à 100 000 URL par lot selon le compte
Site qui bloque les robots	Decodo, Oxylabs, ScrapingBee, Bright Data, Crawlbase
Google Maps, réseaux, marketplaces	Apify et ses scrapers prêts à l'emploi

Serveurs MCP

Accès	Serveurs
Connexion dans `/mcp`	Firecrawl, TinyFish, Olostep, Apify
Sans compte	Playwright, Jina (une partie des outils)
Votre clé	Crawlbase, Decodo, Oxylabs, ScrapingBee, Bright Data

Ces services demandent en général un compte et certains sont payants. Les clés sont stockées en clair dans `~/.copilot/mcp-config.json` : ne partagez pas ce fichier.

Moteur local de gros volumes

- File d'URL en base : reprise exacte après un arrêt
- Limites par site et respect de `robots.txt`
- Nouvel essai automatique sur les erreurs temporaires
- Contenu identique stocké une seule fois, compressé
- Exports NDJSON, CSV et Parquet

Environ 100 pages/s mesurées sur 20 000 pages, contre un serveur local. Sur le vrai web, le débit dépend surtout des limites de chaque site.

Rangement de chaque projet

```
scraping/2026-11-03_catalogue/
├── README.md     méthode, couverture
├── config/       URL, sélecteurs, schéma
├── state.db      file d'URL et résultats
├── raw/          pages brutes, intactes
├── clean/        données validées
├── rejects/      lignes refusées et raison
├── exports/      Parquet, CSV, Excel
├── scripts/      code du projet
└── manifest.json paramètres, compteurs
```

Précision

- Chaque ligne garde sa source et sa date
- Une valeur manquante reste vide, jamais devinée
- Une page de captcha n'est pas comptée comme un succès
- La couverture réelle est annoncée en chiffres

> **Résultats variables selon les cibles.** Au test Proxyway d'octobre 2026 sur des cibles difficiles, Decodo atteint 87 %, Oxylabs 86 %, ScrapingBee 84 % et Firecrawl 34 % de réussite. Aucun outil n'obtient 100 %. L'agent teste donc sur votre cible avant de lancer un gros volume. Il reste aussi dans le cadre légal : données accessibles uniquement, pas de contournement d'identifiant ni de paywall, attention au RGPD pour les données personnelles.

Décider vite et juste

Quand un agent hésite, il ne vous interroge pas : il suit une méthode commune à tous les agents, tranche, puis note sa décision dans le rapport.

Situation	Outil	Coût
Fait, calcul, date	Du code	Gratuit
Choix rapide entre plusieurs options	JEV, par appel direct à TypeSafe	0,042 $ par million de jetons en entrée
Décision lourde, échecs répétés	Rubber duck, critique intégré à Copilot	Crédits Copilot
Problème flou, autres outils indisponibles	`sequential-thinking`, local	Gratuit
Objectif dans un navigateur	Jev Ultrafast, ou Playwright sans clé	Clé TypeSafe (et OpenRouter pour saisir du texte), ou gratuit

> **JEV conseille, l'agent décide.** Son éditeur le dit fort sur les décisions immédiates et faible sur les raisonnements complexes, les nombres et les dates. Les requêtes partent chez TypeSafe, aux États-Unis, dont la politique exclut l'entraînement sur vos requêtes mais ne donne pas de durée de conservation : les agents ne lui envoient ni secret, ni donnée personnelle, ni code privé. Il ne décide jamais du danger d'un script ni de la portée d'un pentest.

Pentester vos applications

`cop-pentest` attaque vos applications, prouve chaque faille par l'exploitation, propose le correctif, rejoue l'attaque et écrit un rapport. La portée comprend les cibles que vous nommez, celles de `pentest-scope.txt` et les cibles locales.

> **Ce qui reste hors du kit.** Le modèle peut refuser certaines demandes et les conditions de GitHub Copilot s'appliquent : aucun réglage ne lève cela. N'attaquez que ce dont vous avez la responsabilité, et préférez WSL, Docker ou une machine virtuelle, car les outils offensifs déclenchent souvent l'antivirus.

Sans Gardien : ce qui n'est plus protégé

> **Rien ne contrôle les commandes de l'IA.** Une page piégée ou un dépôt hostile peut lui faire supprimer des fichiers, voler des mots de passe ou installer un programme. Seuls l'antivirus et les droits de votre compte Windows limitent les dégâts. Gardez la protection en temps réel de Windows activée, ainsi que des sauvegardes de vos données importantes.

`cop` fait confiance au dossier courant sans le vérifier : inspectez un dépôt inconnu avant d'y lancer `cop`. Rien ne bloque non plus l'envoi d'un secret vers JEV.

État des vérifications

État	Élément
Vérifié	Installation et relance, tests du moteur, du serveur Jev et du navigateur Jev, agent `scraper` chargé dans une vraie session Copilot
À confirmer	Délégation réelle entre agents, connexion aux serveurs MCP et appel réel de leurs outils, vrai appel à TypeSafe, rubber duck en autopilot, agent `pentest` en session : ces essais font partie des tests d'installation

Revenir en arrière

Chaque fichier modifié est d'abord sauvegardé dans `~/.copilot/backups/`. Demandez à l'IA : « *Applique la section Retour arrière du kit.* » Pour réduire les droits le temps d'une session, utilisez `/permissions`.

Vérifié avec GitHub Copilot CLI 1.0.105 sous Windows 11, le 3 novembre 2026. Kit version 2.2 sans Gardien. Détails techniques : `docs/REFERENCE_TECHNIQUE.md`.